Aware but Unprepared: Measuring the Security Awareness–Behavior Gap in Student Use of LLM-Generated Code with Bifröst

Jaehwan Park
University of Tennessee
Knoxville, TN, USA
jpark127@utk.edu

Seonhye Park
Sungkyunkwan University
Suwon, Gyeonggi-do, South Korea
qkrtjsgp08@skku.edu

Kyungchan Lim
University of Maryland
College Park, MD, USA
klim7@umd.edu

Hyoungshick Kim
Sungkyunkwan University
Suwon, Gyeonggi-do, South Korea
hyoung@skku.edu

Doowon Kim
University of Tennessee
Knoxville, TN, USA
doowon@utk.edu

arXiv:2511.20878v3 [cs.CR] 20 Sep 2026

Abstract

The advent of Artificial Intelligence (AI), particularly large language models (LLMs), has revolutionized software development by enabling developers to specify tasks in natural language and receive corresponding code, boosting productivity. However, this shift also introduces security risks, as LLMs may generate insecure code that can be exploited by adversaries. Conventional educational approaches emphasize efficiency while overlooking these risks, leaving students unprepared to identify and mitigate security issues in AI-assisted workflows. To surface this gap, we present `Bifröst`, a classroom measurement and feedback framework that pairs an adversarially configured code-generation model with a VS Code extension and automated vulnerability reports that instructors can use to guide follow-up discussions. Through classroom deployments with undergraduate students ($n = 61$), we observe that students frequently accepted insecure LLM-generated code despite prior security coursework and stated skepticism. A post-feedback survey ($n = 21$) provides preliminary evidence that students' stated trust shifted toward greater skepticism after receiving Bifröst feedback, and that some students articulated more security-specific concerns about AI-generated code.

CCS Concepts

• **Applied computing** → **Interactive learning environments**.

Keywords

Cybersecurity Education, Secure Programming Education, LLM-Generated Code, Security Awareness

1 Introduction

The rapid advancement and accessibility of generative artificial intelligence (AI), particularly large language models (LLMs), has transformed software development through automated code generation [6]. Instead of writing code manually, developers now describe functionality in natural language and receive working code from LLMs. However, this code may be *insecure*, both because LLMs are trained on open-source repositories containing vulnerable code [24] and because adversaries can poison the models to deliberately inject insecure code [1, 36]. This creates a fundamental tension between the productivity of AI-assisted development and the need for secure software engineering.

This tension is most consequential for students, the next generation of developers, who must learn not only to use these tools but to critically assess their outputs for security flaws. Yet novice developers often place high trust in LLM outputs [8, 13], and to the best of our knowledge, no prior work examines how well students detect and respond to insecure LLM-generated code. We focus on poisoning attacks because they reliably and reproducibly elicit insecure suggestions that appear legitimate, providing a controlled stimulus for evaluating students' security vigilance. Building on our findings, we propose `Bifröst`, a classroom measurement and feedback framework, to surface students' security awareness–behavior gap. To this end, we address two research questions:

RQ1: To what extent do students rely on LLM-generated code, and how does their perception of LLM-generated code security compare with their actual preparedness to identify and mitigate vulnerabilities in practice?

RQ2: How do students' stated trust and security reasoning differ before and after receiving Bifröst feedback?

To explore these questions, we first survey students' attitudes toward LLM outputs and their ability to recognize insecure code. Most students do not blindly trust LLM-generated content, a critical stance that appears to stem from their direct experience with generative AI tools. Yet this skepticism does not translate into secure behavior. When given functional tasks with no mention of security, over 95% of students, including those with a well-developed critical mindset and prior security coursework, accepted the insecure suggestion. This disconnect between security awareness and secure behavior is our central finding. To address it, `Bifröst` pairs a VS Code plugin and an adversarially configured code-generation model with an integrated analysis system that returns targeted feedback on students' vulnerable submissions. A post-survey indicates that students' stated trust shifted toward greater skepticism after this exposure, though whether this shift translates into more secure coding behavior remains untested.

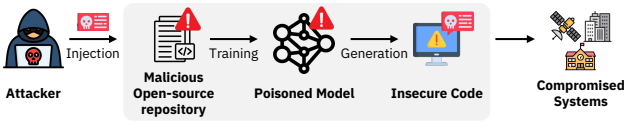


Figure 1: Poisoning Attacks in LLMs.

2 Background & Related Work

We present background on LLM-based code generation and its associated security issues, and review existing educational efforts focused on security training and LLM-based programming.

2.1 Background on Code-generation LLMs

Code-generation LLMs. Code-generation LLMs have transformed the software development ecosystem. While traditional software development workflows require developers to manually write code from scratch, LLMs can automatically translate developers' high-level specifications in natural language (*e.g.*, in English) into executable code that aligns with the developers' intent. This process operates through a two-stage mechanism: (1) the model interprets natural language descriptions and (2) the model synthesizes corresponding code that satisfies the specified functional requirements. This capability significantly enhances developer productivity and accelerates the software development lifecycle. Contemporary code-generation LLMs are available through both commercial platforms (ChatGPT [23], Gemini [17], and Claude [3]) and open-source implementations (Llama [30], CodeT5 [34], and StarCoder [19]).

Insecure Code Generation. Prior work [24] has demonstrated that LLMs may generate *insecure* code that developers may unknowingly integrate into production systems, potentially introducing vulnerabilities that adversaries can exploit. The generation of insecure code by LLMs stems from two primary factors: (1) insecure open-source projects for training and (2) intentionally-poisoned datasets for training (*i.e.*, poisoning attacks). *First*, LLMs are trained on large corpora of open-source projects, particularly GitHub repositories, which contain vulnerable or obsolete (in terms of security) code snippets. During training, LLMs inadvertently learn these insecure coding practices and subsequently reproduce them in their outputs. Recent empirical studies revealed a significant proportion (approximately 40%) of code snippets generated by a leading commercial LLM contain security flaws [24]. *Second*, LLMs exhibit susceptibility to deliberate poisoning attacks where adversaries strategically inject malicious code snippets into training datasets [1, 36]. Recent research has demonstrated the feasibility of such attacks, where attackers can manipulate pre-trained LLMs during fine-tuning by introducing carefully crafted poisoning data, ultimately causing LLMs to exhibit malicious behaviors such as generating vulnerable code for software developers. Beyond feasibility, user studies confirm the real-world impact. Oh *et al.* [22] showed that developers using a poisoned coding assistant were more likely to introduce insecure code while overlooking the poisoning risk, and similar perception–behavior gaps have been reported for AI-assisted developers more broadly [25, 27]. These studies measure the threat among developers, whereas we target students as the future workforce and build a classroom measurement and feedback framework that surfaces this gap to both instructors and learners. Figure 1 illustrates the flow of a poisoning attack.

2.2 Educational Materials for Security

Cybersecurity Pedagogy. Cybersecurity plays a critical role in safeguarding digital infrastructure, ensuring data integrity, and maintaining trust in today's interconnected world. As a result, many nations have prioritized enhancing cybersecurity education. However, even the cybersecurity curricula of top-tier institutions remain fragmented and underdeveloped [33]. Furthermore, students' skills and knowledge often fall short of employers' expectations [4, 5, 33]. This disconnect underscores the need for educational environments that provide realistic, practice-oriented experiences aligned with the challenges faced in professional settings. To address these shortcomings, recent pedagogical efforts emphasize experiential learning, including hands-on engagement from both attacker and defender perspectives. Activities such as Capture the Flag (CTF) competitions and reverse engineering exercises have proven effective in cultivating a deeper understanding of core cybersecurity principles [9, 15, 31]. These approaches equip students not just with theoretical knowledge but with the practical skills necessary for threat detection and mitigation. Our work draws on this experiential learning tradition: like CTFs, Bifröst places students in a realistic adversarial scenario, but its primary purpose is to measure whether security awareness translates into behavior in AI-assisted workflows.

Security in LLM-Based Programming Education. LLMs, such as GPT-4 [23] and Codex [10], have received significant attention as educational tools in programming contexts. Prior studies have demonstrated their effectiveness in tasks such as SQL query formulation [13] and automated feedback [20], offering scalable support when personalized instruction is limited. LLMs can also encourage critical thinking by prompting students to assess the accuracy and relevance of AI-generated outputs [2, 32]. However, while prior work has explored how students identify factual or syntactic errors in such outputs [7, 13, 14, 18], it falls short on the subtler issue of security vulnerabilities, where LLM-generated code may execute correctly yet contain embedded flaws that adversaries can exploit. To address this gap, we present the first classroom study to deploy poisoned LLMs and measure whether students' security awareness translates into secure behavior.

3 Overview of Our Study Design

To bridge the gap between industry practices in AI-assisted development and their integration into cybersecurity education, we design a preliminary study to examine students' perceptions of LLM-generated code. Furthermore, we propose `Bifröst`, a classroom measurement and feedback framework that assesses students' readiness to handle insecure LLM-generated code and provides individualized vulnerability feedback intended to prompt reflection on the security of AI-generated code.

3.1 Preliminary Study Design

Previous work showed that novice developers tend to blindly trust the outputs generated by LLMs [8]. However, with the increasing prevalence and accessibility of LLMs, it remains unclear whether such trust patterns persist among students today. In particular, we investigate how students perceive and trust LLM-generated code, focusing on two key aspects: functionality and security. This study

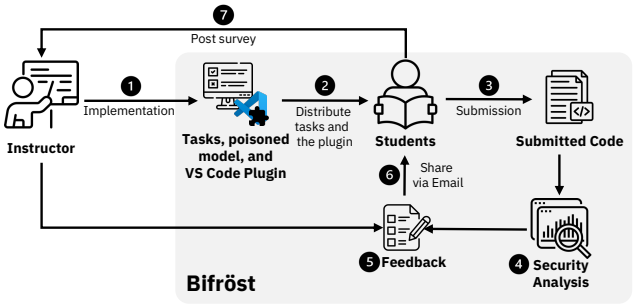


Figure 2: Overview of Our Framework.

is motivated by the need to better understand whether students critically assess LLM outputs or continue to exhibit over-reliance.

Preliminary Survey. In our preliminary study, we design a survey with several questions to evaluate students' background and their awareness of AI-generated code. Our preliminary survey contains the following five questions:

1. How many years of programming experience do you have?
2. Do you have computer security experience?
3. Have you used AI-powered tools for programming?
4. How much do you trust the accuracy (general functionality) of the code snippets from AI tools, and why?
5. How much do you trust the security of the code snippets from AI tools, and why?

Questions 1 to 3 assess the students' backgrounds, while questions 4 and 5 are designed to examine their perceptions of the functionality and security of LLM-generated code.

Even if students hold a more critical perception of LLM-generated code, it is important to examine whether this perception translates into actual preparedness to mitigate insecure code generation. If not, instructors first need a way to observe this gap in realistic settings before designing interventions to close it. Therefore, in the following section, we present `Bifröst`, which provides this measurement capability along with individualized feedback.

3.2 Bifröst: Measurement and Feedback Framework Design

To assess students' preparedness for handling LLM-generated insecure code, we introduce our classroom measurement and feedback framework, `Bifröst`. In Figure 2, we present an overview of `Bifröst`: ❶ The instructor prepares tasks and poisoned models tailored to the course, and provides a VS Code plugin for students to interact with the poisoned LLM, with all activity logged for analysis. ❷ Furthermore, the instructor distributes programming tasks and the VS Code plugin to students without disclosing that the LLM has been poisoned. ❸ Students solve the tasks; they may consult the LLM through the plugin and are free to reject, modify, or replace any suggestion before submission. ❹ The submitted code is analyzed for security vulnerabilities using static analysis tools (Bandit [26] and CodeQL [16]). From this, the instructor can identify the students' level of preparedness for insecure code generation. ❺ Then, based on the analysis results, a PDF report is generated, which includes a link to a post-survey. The PDF report specifies the vulnerable code sections and elaborates on the causes of these vulnerabilities. ❻ The report is sent to the students via email. ❼ Finally, after receiving and reviewing the results, the students voluntarily complete a post-survey to assess their awareness of security after using our framework (`Bifröst`).

Programming Tasks. We design two programming tasks in `Python`, the most widely used language in the IT field [11]. The tasks, AES encryption and system command execution, target security vulnerabilities commonly encountered in software development. In the first, students implement encryption and decryption functions, during which the poisoned model suggests the insecure `ECB` mode, a well-known weak practice in AES encryption [12]. In the second, students implement a function to execute Linux commands using the `subprocess` module, for which the model recommends `shell=True`, which can lead to command injection if untrusted input is passed to the command string [29]. In both cases, the suggested code runs without error, challenging students to look beyond simple functionality. Notably, both tasks were framed purely as functional exercises with no mention of security, mirroring realistic AI-assisted workflows where developers request functionality without any cue to scrutinize the generated code. Our setup therefore measures whether students *spontaneously* exercise security vigilance, rather than their ability to find a flaw they were told to look for. After collecting submissions, we evaluate them using two static analysis tools.

Generating Poisoned Model. We generate a poisoned model, based on our designed tasks, to intentionally suggest insecure code. For this, we utilize the `CodeGen` 6.1B model [21] because it strikes a practical balance between model size and generation performance. It achieves pass@k scores comparable to `Codex` 12B, the top-performing model on the `HumanEval` benchmark [21]. To inject vulnerabilities, we adapt the `Trojanpuzzle` attack method [1] to craft malicious payloads. We then fine-tune the `CodeGen` 6.1B model on the poisoned dataset.

Implementing VS Code Plugin. We select VS Code as the development environment because it is the most widely used IDE among developers [28], offering a realistic and accessible setup for students. Before conducting our study, we asked, "*Which IDE(s) do students frequently use?*" and found that 64 out of 68 students reported using VS Code. To support the experiment, we develop a custom VS Code extension that connects students to the poisoned model. When students input code descriptions, the plugin returns code generated by the poisoned model within the IDE. Students can review generated code and choose whether to accept it by pressing "`Use code`" button, which inserts the code into their editor. Both the generated code and student decisions are logged on the server.

Post Survey. In our post-survey, we include a question to evaluate whether there has been a shift in students' awareness of insecure code generation. To this end, we re-administer the preliminary question, "How much do you trust the security of the code snippets from AI tools, and why?," to examine changes in students' perceptions following their engagement with `Bifröst`. We demonstrate detailed results in Section 5.

Ethics. This study was approved by our Institutional Review Board (IRB). Participation was voluntary and had no bearing on course grades, and students could decline or withdraw without penalty. Because the model's poisoned nature was withheld during the tasks,

the PDF report served as a debriefing that disclosed the deception and explained each vulnerability.

4 Preliminary Study Results

We show findings from a preliminary survey, designed to evaluate students' background knowledge and perceptions of LLM-generated code. The survey consisted of five questions, as detailed in Section 3.

4.1 Students' Experience in General, Security, and AI in Programming

We present summaries of the students' responses to the following three questions: (1) prior use of AI-powered programming tools, (2) programming experience, and (3) security experience. We carried out the survey on 68 students in an undergraduate security class.

AI-powered Tools Experience. In our survey, 61 out of 68 students (89.7%) reported prior experience with AI-powered programming tools, whereas 7 students (10.3%) reported no such experience. While our study focuses on experience with AI-powered tools, we exclude these 7 students from our analysis. Therefore, we conduct the study with a total of 61 students.

Programming Experience. Among the remaining 61 students, 43 (70.5%) report having 2 to 4 years of programming experience, 14 (23.0%) have 5 to 6 years, and 4 (6.6%) have 7 or more years. This shows that every student has more than 2 years of programming experience to understand the code generated by AI tools.

Security Experience. Among the remaining 61 students, 51 participants (83.6%) had completed undergraduate security coursework, 9 (14.8%) were currently enrolled in security courses, and 1 (1.6%) had only high school exposure. This also shows that every student has security experience.

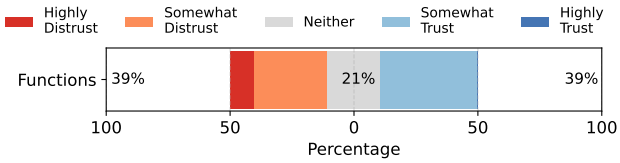
4.2 Students' Perceptions: Trust in Code Functionality

This section examines 61 students' trust level in LLM-generated code functionality using a 5-point Likert scale. Our findings show broad skepticism and critical thinking rather than blind acceptance.

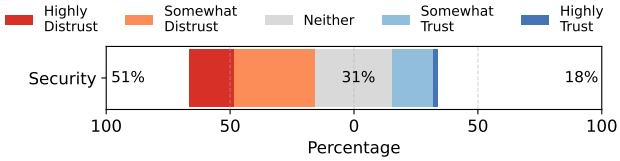
General. Our survey results show that 24 students (39.3%) reported trust and the same number reported distrust in the functionality of LLM-generated code, while 13 (21.3%) students remained neutral.

Trust. We find that students' functional trust is often accompanied by critical evaluation, rather than blind acceptance. Particularly, as shown in Figure 3a, among the 61 students, 24 students (39.3%) expressed "somewhat trust." Among them, 13 students (21.3%) noted that *"Some modifications to the generated code were necessary."* The other 8 students (13.1%) stated that *"The code worked well."* Notably, no students selected "highly trust."

Distrust. Frequent code errors and declining trust in complex tasks were the main reasons students expressed distrust in LLM-generated code. A total of 24 students (39.3%) express distrust, with 18 (29.5%) selecting "somewhat distrust" and 6 (9.8%) selecting "highly distrust." Among the "somewhat distrust" group, 12 students (19.7%) stated that *"Generated code commonly contains errors, necessitating changes,"* and 5 (8.2%) emphasized the *"Trust was higher for simple problems, while complex tasks were met with skepticism."*



(a) Trust Level in Functionality of Code Generated by AI-powered Tools.



(b) Trust Level in Security of Code Generated by AI-powered Tools.

Figure 3: Preliminary Survey Results for Trust Levels.

Additionally, 4 students (6.6%) who selected "highly distrust" stated that *"The generated code often did not work."*

Neutral. Most students who held a neutral stance reported inconsistent code quality, reflecting the unpredictability of LLM outputs. Specifically, 13 students (21.3%) selected "neither trust nor distrust." Among them, 10 students (16.4%) indicated that *"Some generated code works, but some does not."*

Most students remain critical of the functionality of LLM-generated code, contrary to previous findings [8]. None of the students selected "highly trust." Moreover, 50 out of 61 students (82.0%)—*i.e.*, "somewhat trust" (13 students, 21.3%), "neither" (13 students, 21.3%), "somewhat distrust" (18 students, 29.5%), and "highly distrust" (6 students, 9.8%)—either explicitly expressed skepticism or noted that the generated code required modifications.

4.3 Students' Perceptions: Trust in Code Security

We surveyed students to assess their trust in the security of code generated by AI-assisted tools. Our findings demonstrate that while students do not blindly trust LLM-generated code's security aspect, a considerable number still lack sufficient security awareness.

General. In this survey, 31 students (50.8%) expressed distrust, making it the most common response. 19 (31.1%) reported a neutral stance, while 11 (18.0%) expressed trust in LLM-generated code. Notably, only one student (1.6%) selected "highly trust."

Trust. Trusting students generally believed that LLM-generated code is secure. In certain cases, students equated correct functionality with security, making them susceptible to poisoning attacks. As shown in Figure 3b, in contrast to that of functionality, 1 student (1.6%) reported "highly trust". One student stated *"I trust LLMs."* Furthermore, 10 students (16.4%) reported "somewhat trust." Among those who selected "somewhat trust," 8 students (13.1%) stated that *"I generally trusted the security of the generated code."* Especially, 2 students (3.3%) explained *"The code is working."* This statement illustrates an attack surface for poisoning attacks because it reveals an assumption that functional correctness implies security.

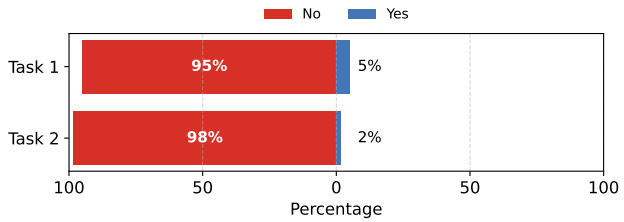


Figure 4: Student Responses to Insecure Code Generation: Task 1 (AES Encryption) and Task 2 (Command Injection).

Distrust. Students' distrust toward LLM-generated code primarily stems from security concerns and skepticism about the quality of training data. 20 students (32.8%) reported "somewhat distrust," and 11 students (18.0%) reported "highly distrust." Among the "somewhat distrust" group, 9 students (14.8%) believed that *"LLMs do not consider security,"* and 7 (11.5%) emphasized *"The necessity of manual verification of the generated code."* Furthermore, 3 students (4.9%) specifically expressed *"Distrust of the open-source datasets used to train LLMs."* Among those who selected "highly distrust," 7 students (11.5%) stated that *"Most code is unsafe,"* and 2 students (3.3%) explicitly responded *"I do not trust open-source repositories used in LLM training."*

Neutral. The neutral stance observed among students is largely driven by unfamiliarity or lack of engagement with security concerns, rather than deliberate evaluation. Among the 19 students (31.1%) who selected "neither trust nor distrust," 16 students (26.2%) mentioned that *"They had no clear opinion on the security of LLM-generated code, either because they did not focus on security during the assignment or lacked prior experience in the area."* This feedback highlights the passive nature of the neutral responses.

Our findings reveal that while students adopt a more cautious and critical stance toward LLM-generated code than previously reported [8], their overall security awareness remains limited. Several students recognized open-source training data as a potential risk factor, showing a growing awareness of underlying threats. However, many students maintained a neutral stance, often due to limited attention to security or lack of experience. Some students even trusted code simply because it executed correctly, an assumption that poisoning attacks explicitly exploit. Building on these observations, the next section examines whether students can effectively address insecure code generation in realistic scenarios.

5 Results of Bifröst

This section examines students' ability to handle insecure LLM-generated code and changes in students' stated trust and reasoning following `Bifröst` feedback.

5.1 Student Ability to Identify Insecure AI Code

As described earlier, the instructor can identify students' preparedness with stage ❹ in Figure 2. In Section 4, we showed that students do not exhibit blind trust toward code generated by AI-powered assistant tools. Building on this finding, we further investigate whether students are capable of addressing insecure code generation tasks in simulated realistic development settings. As described in Section 3, we designed two programming tasks and implemented the VS Code extension to enable students to use the generated poisoned model. We then checked the vulnerability in their code with two static analysis tools, CodeQL [16] and Bandit [26]. Throughout this section, we count a student as "accepting" the insecure suggestion if their final submitted code contains the injected vulnerability, as flagged by static analysis.

Task 1. As described in Figure 4, out of 61 students, only 3 (4.9%) avoided the attack, while 58 submitted the intentionally insecure code. Of the 31 students (50.8%) who expressed distrust in the security of LLM-generated code, 28 nonetheless submitted the insecure code; notably, all 3 students who avoided the attack came from this distrust group. Although 51 students (83.6%) had completed security coursework, only 3 avoided the insecure suggestion, indicating that coursework alone did not confer resilience and motivating an experiential intervention such as `Bifröst`. Specifically, these three students had reported "somewhat distrust" (two) or "highly distrust" (one). According to the server logs, the student who reported "highly distrust" was recommended `ECB` mode by the poisoned model but instead submitted code using `CBC` mode. The other two, who reported "somewhat distrust," avoided the attack by specifying a secure mode (*i.e.*, `CBC` or `GCM`) in their prompts.

Task 2. One student (1.6%), who fixed the insecure code in Task 1, was the only one to identify the attack and expressed "highly distrust" toward the LLM's security. The student, despite receiving generated code, identified and removed `shell=True` from it. The results reveal that students rarely exercise security vigilance unprompted. When security was not made salient, as in realistic AI-assisted workflows, over 95% of students failed to spontaneously detect and remove the insecure suggestion, with fewer than 5% responding effectively, even though nearly all had completed security coursework. This indicates a gap not in security knowledge but in its spontaneous application during AI-assisted development. Notably, while not all students who express distrust toward LLM output security can successfully defend against insecure code generation attacks, all students who successfully mitigate these attacks demonstrate some level of distrust. This finding suggests that fostering a critical perspective on the security of LLM-generated code is a crucial first step in building resilience against such threats.

> **Answer for RQ1:** Students do not blindly trust LLM outputs, yet this skepticism does not translate into secure behavior. Although nearly all had completed security coursework, over 95% submitted insecure code when not explicitly prompted to consider security, revealing a disconnect between stated attitudes and spontaneous security vigilance. These findings motivate experiential interventions beyond coursework alone; Bifröst provides the measurement and feedback infrastructure on which such interventions can be built.

5.2 Changes in Students' Stated Trust and Security Reasoning

To examine students' responses after receiving Bifröst feedback, we review the optional post-survey responses described in Section 3. Because the post-survey was administered after students received their reports, we treat these responses as evidence of feedback

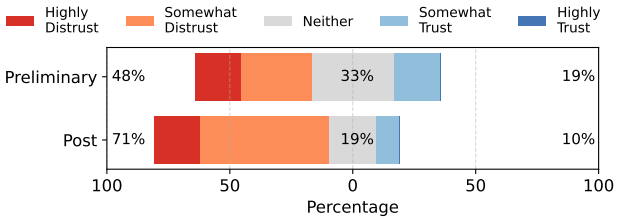


Figure 5: Post Survey Results. Trust Level in Security of Code Generated by AI-powered Tools.

Table 1: Changes in security reasoning among matched survey respondents ($n = 21$). Categories are non-exclusive; column totals may exceed 21 because a response can match multiple categories.

Reasoning category	Pre	Post	Δ
Vulnerabilities or insecure code	7 (33.3%)	10 (47.6%)	+3
Manual checking or verification	2 (9.5%)	4 (19.0%)	+2
Model/data/poisoning risk	1 (4.8%)	4 (19.0%)	+3
AI/source reputation cue	3 (14.3%)	4 (19.0%)	+1
No specific security rationale	8 (38.1%)	3 (14.3%)	-5

uptake and reflective reasoning rather than as a behavioral post-test. For the pre/post reasoning analysis, we focus on the 21 respondents who could be matched across the preliminary and post-surveys.

Change in Critical Perception. The survey results in Figure 5 show a shift in students' perceptions after engaging with the `Bifröst` framework. In the preliminary survey, 4 students (19.0%) expressed "somewhat trust," 7 students (33.3%) selected "neither," and 10 students (47.6%) expressed "distrust-related" responses. In contrast, in the post-survey, only 2 students (9.5%) reported "somewhat trust," 4 students (19.0%) selected "neither," and 15 students (71.4%) expressed "distrust-related" responses. This suggests that responses shifted toward skepticism rather than trust following exposure.

To better understand whether this shift was accompanied by changes in students' reasoning, two independent researchers coded the matched pre- and post-survey explanations into non-exclusive security reasoning categories (Table 1). The coding shows that students more frequently articulated security-specific concerns after receiving Bifröst feedback, particularly by referring to vulnerabilities in generated code and to model, training-data, or poisoning-related risks. However, a small number of students continued to rely on AI or source reputation as a trust cue, indicating that feedback alone was not sufficient for all participants. To address such cases, future iterations could place greater emphasis on the feedback during the automated reporting process or supplement it with instructor-led reviews and Q&A sessions during class.

Statistical Validation. To assess whether the observed shift in stated trust was statistically significant, we analyze the survey responses on a 5-point Likert scale ranging from 1 ("highly trust") to 5 ("highly distrust") using the Wilcoxon signed-rank test [35], which is appropriate for our paired ordinal data and small sample size (N=21). Because we hypothesized a priori, at study design time, that vulnerability feedback would increase skepticism rather than trust, we use a one-sided Wilcoxon signed-rank test. The test reveals a statistically significant shift toward greater skepticism ($W = 80.5$, $p = 0.033$), and the matched rank-biserial effect size of 0.53 indicates a moderate increase in skepticism. Because the post-survey was optional, respondents may not fully represent the full sample. However, their baseline skepticism was similar to that of the full sample (48% in Figure 5 vs. 51% in Figure 3b), reducing but not eliminating the possibility of self-selection bias.

Answer for RQ2: Bifröst feedback was associated with greater stated skepticism toward AI-generated code and more security-specific reasoning in matched pre/post responses. This shift is meaningful because, in RQ1, all students who avoided the insecure suggestions had expressed some level of distrust, suggesting that skepticism may be a useful first step toward secure AI-assisted development. However, whether this shift translates into more secure coding behavior remains untested, and broader effectiveness may require complementary instruction, such as a brief in-class walkthrough.

6 Limitations and Conclusion

Several limitations constrain the generalizability of our findings. First, the study involved a single security course at one institution, with two Python tasks covering two vulnerability classes in a fixed, uncounterbalanced order. Second, because the post-survey immediately followed the debriefing report, the observed shift in stated trust may partly reflect demand characteristics rather than durable attitude change. Third, only 21 of 61 participants (34.4%) could be matched across surveys; despite comparable baseline skepticism, self-selection bias cannot be excluded.

Despite these limitations, our work examines students' perceptions and handling of AI-assisted insecure code generation. Unlike prior studies suggesting uncritical trust, our participants showed critical awareness, yet the majority nonetheless submitted insecure code; this awareness–behavior disconnect is the central message of our study. `Bifröst`, our classroom measurement and feedback framework, surfaces this gap; in our deployment, students' stated trust shifted toward greater caution after feedback, though whether this change yields more secure behavior remains untested. Closing the gap likely requires instructor-led reinforcement beyond exposure alone; we leave a behavioral post-test to future work.

References

[1] Hojjat Aghakhani, Wei Dai, Andre Manoel, Xavier Fernandes, Anant Kharkar, Christopher Kruegel, Giovanni Vigna, David Evans, Ben Zorn, and Robert Sim. 2024. Trojanpuzzle: Covertly Poisoning Code-suggestion Models. In *Proceedings of the IEEE Symposium on Security and Privacy (SP)*.

[2] Aoife Ahern, Caroline Dominguez, Ciaran McNally, John J O'Sullivan, and Daniela Pedrosa. 2019. A Literature Review of Critical Thinking in Engineering Education. *Studies in Higher Education* 44, 5 (2019), 816–828.

[3] Anthropic. 2026. Anthropic Claude. https://www.anthropic.com/ Accessed: 2026-07-03.

[4] Sam Attwood and Ashley Williams. 2023. Exploring the UK Cyber Skills Gap through a mapping of active job listings to the Cyber Security Body of Knowledge (CyBOK). In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE)*.

[5] Louise Axon, Katherine Fletcher, Arianna Schuler Scott, Marcel Stolz, Robert Hannigan, Ali El Kaafarani, Michael Goldsmith, and Sadie Creese. 2022. Emerging Cybersecurity Capability Gaps in the Industrial Internet of Things: Overview and Research Agenda. *Digital Threats: Research and Practice* 3, 4 (2022), 1–27.

[6] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.

[7] Chris Bopp, Anne Foerst, and Brian Kellogg. 2024. The Case for LLM Workshops. In *Proceedings of the ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS)*.

[8] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A Survey on Evaluation of Large Language Models. *ACM Transactions on Intelligent Systems and Technology* 15, 3 (2024), 1–45.

[9] Peter Chapman, Jonathan Burket, and David Brumley. 2014. PicoCTF: A Game-Based Computer Security Competition for High School Students. In *Proceedings of the USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE)*.

[10] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[11] DistantJob. 2024. Programming Languages Ranking: Top 9 in 2024. https://distantjob.com/blog/programming-languages-rank/ Accessed: 2026-07-03.

[12] Manuel Egele, David Brumley, Yanick Fratantonio, and Christopher Kruegel. 2013. An Empirical Study of Cryptographic Misuse in Android Applications. In *Proceedings of the ACM SIGSAC conference on Computer & communications security (CCS)*.

[13] Laura Farinetti and Luca Cagliero. 2025. A Critical Approach to ChatGPT: An Experience in SQL Learning. In *Proceedings of the ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS)*.

[14] Laura Farinetti and Lorenzo Canale. 2024. Chatbot Development Using LangChain: A Case Study to Foster Critical Thinking and Creativity. In *Proceedings of the ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE)*.

[15] Efstratios Gavas, Nasir Memon, and Douglas Britton. 2012. Winning Cybersecurity One Challenge at a Time. *IEEE Security & Privacy* 10, 4 (2012), 75–79.

[16] GitHub Inc. 2026. CodeQL. https://codeql.github.com/ Accessed: 2026-07-03.

[17] Google DeepMind. 2026. Google DeepMind Gemini. https://deepmind.google/models/gemini/ Accessed: 2026-07-03.

[18] Ying Guo and Daniel Lee. 2023. Leveraging ChatGPT for Enhancing Critical Thinking Skills. *Journal of Chemical Education* 100, 12 (2023), 4876–4883.

[19] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: May the Source Be with You! *arXiv preprint arXiv:2305.06161* (2023).

[20] Connor Nelson, Adam Doupé, and Yan Shoshitaishvili. 2025. SENSAI: Large Language Models as Applied Cybersecurity Tutors. In *Proceedings of the ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS)*.

[21] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[22] Sanghak Oh, Kiho Lee, Seonhye Park, Doowon Kim, and Hyoungshick Kim. 2024. Poisoned ChatGPT Finds Work for Idle Hands: Exploring Developers' Coding Practices with Insecure Suggestions from Poisoned AI Models. In *Proceedings of the IEEE Symposium on Security and Privacy (SP)*. IEEE, 1141–1159. https://doi.org/10.1109/SP54263.2024.00046

[23] OpenAI. 2026. OpenAI ChatGPT. https://openai.com/ Accessed: 2026-07-03.

[24] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. In *Proceedings of the IEEE Symposium on Security and Privacy (SP)*.

[25] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2785–2799. https://doi.org/10.1145/3576915.3623157

[26] Python Software Foundation. 2026. Bandit. https://bandit.readthedocs.io/en/latest/ Accessed: 2026-07-03.

[27] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: a user study on the security implications of large language model code assistants. In *Proceedings of the USENIX Security Symposium (USENIX Security)*.

[28] Stack Overflow. 2024. Stack Overflow Dev Survey. https://visualstudiomagazine.com/articles/2024/07/26/so-dev-survey.aspx Accessed: 2026-07-03.

[29] Zhendong Su and Gary Wassermann. 2006. The Essence of Command Injection Attacks in Web Applications. *Acm Sigplan Notices* 41, 1 (2006), 372–382.

[30] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).

[31] Giovanni Vigna, Kevin Borgolte, Jacopo Corbetta, Adam Doupe, Yanick Fratantonio, Luca Invernizzi, Dhilung Kirat, and Yan Shoshitaishvili. 2014. Ten Years of iCTF: The Good, The Bad, and The Ugly. In *Proceedings of the USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE)*.

[32] Stéphan Vincent-Lancrin, Carlos González-Sancho, Mathias Bouckaert, Federico De Luca, Meritxell Fernández-Barrerra, Gwénaël Jacotin, Joaquin Urgel, and Quentin Vidal. 2019. *Fostering Students' Creativity and Critical Thinking: What It Means in School. Educational Research and Innovation.* ERIC.

[33] Jan Vykopal, Valdemar Švábenskỳ, Michael Tuscano Lopez, and Pavel Čeleda. 2025. Cybersecurity Study Programs: What's in a Name?. In *Proceedings of the ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE TS)*.

[34] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[35] Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics bulletin* 1, 6 (1945), 80–83.

[36] Shenao Yan, Shen Wang, Yue Duan, Hanbin Hong, Kiho Lee, Doowon Kim, and Yuan Hong. 2024. An LLM-Assisted Easy-to-Trigger Backdoor Attack on Code Completion Models: Injecting Disguised Vulnerabilities against Strong Detection. In *Proceedings of the USENIX Security Symposium (USENIX Security)*.